
Discounted Beta-Bernoulli Reward Estimation for Sample-Efficient Reinforcement Learning with Verifiable Rewards

Haechan Kim^{1 2} Soohyun Ryu¹ Gyouk Chu¹ Doohyuk Jang¹ Eunho Yang^{1 3}

Abstract

Reinforcement learning with verifiable rewards (RLVR) has emerged as an effective post-training paradigm for improving the reasoning capabilities of large language models. However, existing group-based RLVR methods often suffer from severe sample inefficiency. This inefficiency stems from reliance on point estimation of rewards from a small number of rollouts, leading to high estimation variance, variance collapse, and ineffective utilization of generated responses. In this work, we reformulate RLVR from a statistical estimation perspective by modeling rewards as samples drawn from a policy-induced distribution and casting advantage computation as the problem of estimating the reward distribution from finite data. Building on this view, we propose **Discounted Beta-Bernoulli (DBB)** reward estimation, which leverages historical reward statistics for the non-stationary distribution. Although biased, the resulting estimator exhibits reduced and stable variance and theoretically avoids estimated variance collapse. Under mild non-stationarity, it also achieves a lower mean squared error than standard point estimation, as we characterize analytically and verify empirically. Across six in-distribution and three out-of-distribution reasoning benchmarks, GRPO with DBB consistently outperforms naive GRPO and strong recent baselines, including the replay-based RePO and the variance-collapse-aware GRESO and DAPO. Relative to GRPO, it achieves average Acc@8 improvements of 3.43/2.32 points in-distribution and 10.05/8.34 points out-of-distribution on the 1.7B and 8B models, respectively, without additional computational cost or memory usage.

Under mild non-stationarity, it also achieves a lower mean squared error than standard point estimation, as we characterize analytically and verify empirically. Relative to GRPO, it achieves average Acc@8 improvements of 3.43/2.32 points in-distribution and 10.05/8.34 points out-of-distribution on the 1.7B and 8B models, respectively, without additional computational cost or memory usage.

1. Introduction

RLVR (Lambert et al., 2025) has recently emerged as a key post-training paradigm for enhancing the complex reasoning capabilities of large language models (Plaat et al., 2025) and improving their performance on downstream tasks (Guo et al., 2025; Yang et al., 2025; Yu et al., 2025b). Since most RLVR algorithms, including Group Relative Policy Optimization (GRPO; Shao et al., 2024), rely on group-based advantage estimation (Yu et al., 2025a; Liu et al., 2025; Zheng et al., 2025a; Zhao et al., 2025), they require generating multiple responses per prompt, which can account for a large fraction of the overall training time, often approaching 50% (Le et al., 2025). Despite this substantial computational cost, many existing group-based RLVR algorithms fail to utilize information from the generated responses efficiently.

This sample inefficiency can be attributed to two fundamental characteristics of RLVR. The first characteristic is that the variance of rewards can collapse to zero when all generated responses receive identical rewards under group-relative estimation. This *variance collapse* issue not only wastes the computational cost of response generation but also eliminates meaningful training signals within the batch (Zhang et al., 2025). To address this issue, approaches such as dynamic sampling (Yu et al., 2025a) and GRPO with Efficient Selective Rollout (GRESO; Zheng et al., 2025b) have been proposed. However, dynamic sampling requires several times more rollout cost, and GRESO cannot fully resolve the problem since it probabilistically filters prompts that are likely to result in variance collapse.

The second characteristic is that, due to the on-policy nature of RLVR algorithms, information from all generated responses is discarded after a single gradient update. Recent replay-based methods (Li et al., 2025; Zhan et al., 2025)

¹Kim Jaechul Graduate School of AI, KAIST, Daejeon, South Korea ²KRAFTON, Seoul, South Korea ³AITRICS, Seoul, South Korea. Correspondence to: Eunho Yang <eunhoy@kaist.ac.kr>.

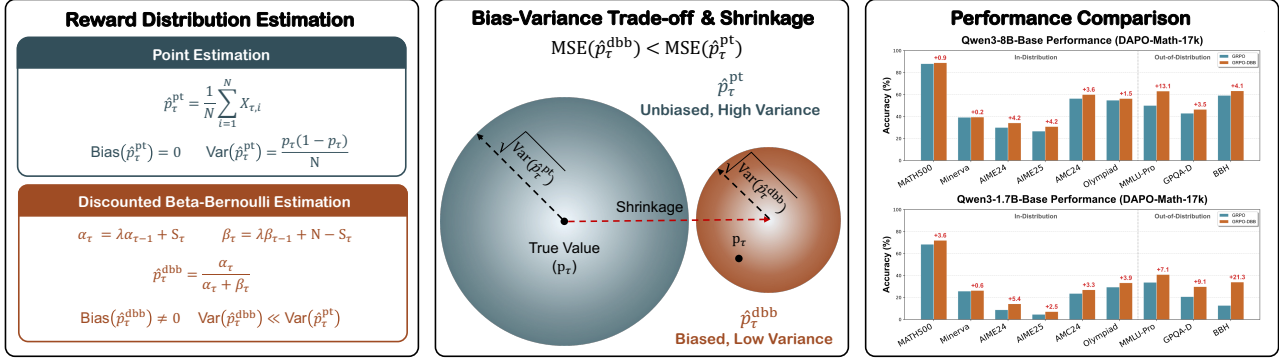


Figure 1. Comparison between point estimation and DBB estimation. By trading a small bias for substantial variance reduction via shrinkage, DBB estimation achieves lower mean squared error. Compared to naive GRPO using point estimation, GRPO with DBB estimation consistently demonstrates superior performance across all benchmarks and both model scales.

attempt to reuse rollouts generated from the training history. However, unbiased reuse of off-policy data requires importance sampling, which in turn necessitates storing all token-level probabilities under historical policies and performing additional forward passes on the current policy. As a result, replay-based approaches introduce substantial GPU memory overhead and additional computational cost, limiting their practical scalability.

These two sources of sample inefficiency primarily arise because most group-based RLVR algorithms rely on point estimators that consider only the rewards from the current rollout group when estimating the underlying reward distribution for advantage estimation. While such estimators can be reliable when the number of sampled responses per group is sufficiently large (Casella & Berger, 2024), practical RLVR settings often operate in low-sample regimes due to computational constraints. In these settings, point estimators remain unbiased but suffer from high variance, which can lead to variance collapse and unstable training dynamics.

To address this limitation, we adopt a statistical perspective that models rewards as stochastic outcomes drawn from a distribution induced by the policy. Instead of relying on point estimation, we employ a Bayesian framework (Gelman et al., 1995) to model uncertainty explicitly and incorporate temporal reward dynamics. Within this framework, we propose the **Discounted Beta-Bernoulli (DBB)** reward estimation, which tracks the evolving reward distribution by discounting historical observations. While DBB introduces a small bias, it significantly reduces variance and avoids variance collapse, thereby providing more stable and informative training signals (Figure 1). Empirically, DBB yields a lower mean-squared error with respect to the true reward distribution than point estimation in low-sample scenarios, provided that the discount factor λ is chosen in a reasonable range.

We evaluate GRPO with the DBB reward estimator (GRPO-DBB) on two model scales, Qwen3-1.7B-Base and Qwen3-8B-Base (Yang et al., 2025), across six in-distribution mathematical reasoning benchmarks, including MATH500, Minerva, AIME24/25, AMC24, and OlympiadBench. GRPO-DBB consistently outperforms naive GRPO and other baselines across all benchmarks and model sizes. Specifically, compared to GRPO, GRPO-DBB achieves average Acc@8 improvements of 3.43 and 2.32 points on Qwen3-1.7B-Base and Qwen3-8B-Base, respectively. These improvements are statistically significant across nearly all benchmarks over four independent runs. Furthermore, on three out-of-distribution reasoning benchmarks such as MMLU-Pro, GPQA-Diamond, and Big-Bench Hard, GRPO-DBB yields average Acc@8 gains of 10.05 and 8.34 points over GRPO on 1.7B and 8B models, respectively. GRPO-DBB also outperforms recent strong baselines including the replay-based RePO and the variance-collapse-aware GRESO and DAPO.

2. Preliminaries

We briefly introduce RLVR and its commonly used baseline, GRPO, to establish the foundation for our method.

2.1. Reinforcement Learning with Verifiable Rewards

Given a prompt $q \sim \mathcal{D}$ from the training dataset and a policy π_θ , a generated response is $o \sim \pi_\theta(\cdot | q)$. In RLVR, the reward function $r(\cdot)$ is typically defined as a binary signal indicating whether the response contains a correct answer.

$$r(o, q) = \begin{cases} 1, & \text{if } o \text{ contains the answer of } q \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

2.2. Group Relative Policy Optimization (GRPO)

GRPO (Shao et al., 2024) is a foundational baseline for RLVR that eliminates the need for an explicit value model

and Generalized Advantage Estimation (Schulman et al., 2017). Instead, it relies on group-relative normalization to estimate advantages.

For a given prompt q , GRPO samples N independent responses $\{o_i\}_{i=1}^N$ from the old policy $\pi_{\theta_{\text{old}}}$ and computes advantages by normalizing the corresponding rewards $\{r(o_i, q)\}_{i=1}^N$ within the group:

$$\hat{A}_i = \frac{r(o_i, q) - \mu}{\sigma}, \quad (2)$$

where $\mu = \frac{\sum_{i=1}^N r(o_i, q)}{N}$ and $\sigma^2 = \frac{\sum_{i=1}^N (r(o_i, q) - \mu)^2}{N-1}$. The resulting advantage $\hat{A}_i$ is broadcast to all tokens in the response, i.e., $\hat{A}_{i,t} = \hat{A}_i$ for all token positions t .

The policy is then updated by maximizing a clipped surrogate objective based on estimated advantages:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{q \sim \mathcal{D}, \{o_i\}_{i=1}^N \sim \pi_{\theta_{\text{old}}}(\cdot|q)} \left[\frac{1}{N} \sum_{i=1}^N \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \min \left(w_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(w_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_{i,t} \right) \right], \quad (3)$$

where ϵ is the clipping hyperparameter and $w_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}|q, o_{i,<t})}$ denotes the per-token importance weight. Following DAPO (Yu et al., 2025a), we omit the KL divergence term between the online and reference policies, as it may restrict exploration.

3. Method

Many group-based RLVR methods (Shao et al., 2024; Liu et al., 2025; Xie et al., 2025; Le et al., 2025) primarily focus on the design of advantage estimators, while paying comparatively little attention to the more fundamental problem of reward estimation that underlies advantage computation. In practice, computational and memory constraints limit the number of sampled responses per prompt, making accurate reward estimation inherently difficult.

In contrast, we introduce a new perspective that reformulates RLVR through the lens of statistical estimation (Section 3.1). Specifically, we view the reward not as a deterministic signal, but as a distribution induced by the policy, and frame advantage computation as a problem of estimating this distribution from finite samples. Building on this reformulation, we propose **Discounted Beta-Bernoulli (DBB)** reward estimation for RLVR, which estimates the non-stationary reward distribution by leveraging historical rewards (Section 3.2). Despite introducing bias, DBB reduces estimator variance and achieves lower MSE. Moreover, it fundamentally prevents variance collapse and preserves informative training signals (Section 3.3).

3.1. Reward Estimation as Distributional Inference

For a given prompt $q \sim \mathcal{D}$ and training step η with corresponding epoch τ , each response generated by the policy $\pi_{\theta_{\eta, \text{old}}}$ produces a binary reward indicating whether the response is correct. Under our formulation, this reward is naturally modeled as a Bernoulli random variable

$$X_{\tau, i} \sim \text{Bernoulli}(p_{\tau}), \quad (4)$$

where $p_{\tau} = \mathbb{P}(X_{\tau, i} = 1 | q, \pi_{\theta_{\eta, \text{old}}})$ denotes the probability of obtaining a correct response.

Within this framework, group-based RLVR algorithms such as GRPO and Dr.GRPO can be interpreted as implicitly estimating the reward distribution from a finite set of rollout samples. Given N observed rewards $\{X_{\tau, i}\}_{i=1}^N$, existing approaches rely on *point estimation* of p_{τ} via the empirical mean

$$\hat{p}_{\tau}^{\text{pt}} = \frac{1}{N} \sum_{i=1}^N X_{\tau, i}. \quad (5)$$

The corresponding reward variance is then estimated using the sample variance,

$$\widehat{\text{Var}}^{\text{pt}}(X_{\tau}) = \frac{\sum_{i=1}^N (X_{\tau, i} - \hat{p}_{\tau}^{\text{pt}})^2}{N-1} = \frac{N \cdot \hat{p}_{\tau}^{\text{pt}} (1 - \hat{p}_{\tau}^{\text{pt}})}{N-1}. \quad (6)$$

Since the estimated variance $\widehat{\text{Var}}^{\text{pt}}$ is determined by the point estimator (mean) $\hat{p}_{\tau}^{\text{pt}}$, it suffices to consider only the point estimator in our analysis. Concretely, the point estimator $\hat{p}_{\tau}^{\text{pt}}$ has expectation and variance as follows:

$$\mathbb{E}[\hat{p}_{\tau}^{\text{pt}} | p_{\tau}] = p_{\tau}, \quad (7)$$

$$\text{Var}(\hat{p}_{\tau}^{\text{pt}} | p_{\tau}) = \frac{p_{\tau}(1 - p_{\tau})}{N}. \quad (8)$$

While the point estimator is unbiased, its variance grows as the number of rollouts N decreases. As a result, individual rollout outcomes can exert disproportionate influence on the estimate, causing the estimator to fluctuate widely. Moreover, the estimated variance can collapse to zero when all sampled rewards are identical. Since group-relative methods compute advantages directly from these reward estimates, such estimation noise naturally propagates to the advantage signal, leading to unstable policy updates.

3.2. Discounted Beta-Bernoulli Reward Estimation

To estimate the Bernoulli reward distribution beyond empirical averaging, we adopt a well-established Bayesian perspective that leverages the conjugacy between the Beta prior and the Bernoulli likelihood. In this setting, the reward probability is modeled using a Beta prior and updated via a Beta posterior after observing rollout outcomes, leading to the naive Beta-Bernoulli model defined below.

Algorithm 1 Discounted Beta-Bernoulli Reward Estimation

input LLM policy π_θ , training dataset $\mathcal{D}$

- 1: Initialize $(\alpha_0^q, \beta_0^q) \leftarrow (1, 1) \quad \forall q \in \mathcal{D}$
- 2: **for** epoch $\tau = 1, \dots, I$ **do**
- 3: **for** iteration $= 1, \dots, M$ **do**
- 4: Sample a minibatch $\mathcal{D}_b \subset \mathcal{D}$
- 5: Set old policy $\pi_{\theta_{\text{old}}} \leftarrow \pi_\theta$
- 6: **for** each prompt $q \in \mathcal{D}_b$ **do**
- 7: Sample outputs $\{o_i\}_{i=1}^N \stackrel{\text{i.i.d.}}{\sim} \pi_{\theta_{\text{old}}}(\cdot | q)$
- 8: Compute rewards $\{X_i = r(o_i, q)\}_{i=1}^N$
- 9: Update posterior parameters:
- 10:
$$\alpha_\tau^q \leftarrow \lambda \alpha_{\tau-1}^q + \sum_{i=1}^N X_i,$$
- 11:
$$\beta_\tau^q \leftarrow \lambda \beta_{\tau-1}^q + N - \sum_{i=1}^N X_i$$
- 12: Compute advantage $\hat{A}_i$ with $(\alpha_\tau^q, \beta_\tau^q)$ and X_i
- 13: **end for**
- 14: **for** update $= 1, \dots, U$ **do**
- 15: Update π_θ by maximizing the objective (Eq. 3)
- 16: **end for**
- 17: **end for**

Definition 1 (Beta-Bernoulli Reward Model). *To model uncertainty in the Bernoulli reward distribution, we place a Beta prior over the reward probability:*

$$p_\tau \sim \text{Beta}(\alpha_\tau, \beta_\tau), \quad (9)$$

where $(\alpha_\tau, \beta_\tau)$ denote the posterior parameters for a prompt at epoch τ , with initialization $\alpha_0 = \beta_0 = 1$.

Given N rollouts with S_τ successes, the posterior distribution is given by:

$$\alpha_\tau = \alpha_{\tau-1} + S_\tau, \quad \beta_\tau = \beta_{\tau-1} + N - S_\tau. \quad (10)$$

However, the above model assumes a stationary reward distribution. In RLVR, this assumption does not hold: due to the on-policy nature of training, the policy π_θ evolves over time, inducing a non-stationary reward distribution. As a result, historical observations may become outdated and should not be weighted equally with recent rollouts.

To address this challenge, we introduce the **Discounted Beta-Bernoulli (DBB)** reward model, which gradually discounts past information to estimate the reward distribution accurately.

Definition 2 (Discounted Beta-Bernoulli Reward Model). *Given posterior parameters $(\alpha_{\tau-1}, \beta_{\tau-1})$ and N rollouts with S_τ successes at epoch τ , the update is defined as*

$$\alpha_\tau = \lambda \alpha_{\tau-1} + S_\tau, \quad \beta_\tau = \lambda \beta_{\tau-1} + N - S_\tau, \quad (11)$$

where the discount factor $\lambda \in (0, 1]$ controls the influence of historical observations.

Under the DBB reward model, we estimate the mean and variance of the Bernoulli reward distribution as follows:

$$\hat{p}_\tau^{\text{dbb}} = \frac{\alpha_\tau}{\alpha_\tau + \beta_\tau}, \quad (12)$$

$$\widehat{\text{Var}}^{\text{dbb}}(X_\tau | \alpha_\tau, \beta_\tau) = \frac{\alpha_\tau \beta_\tau}{(\alpha_\tau + \beta_\tau)^2}. \quad (13)$$

To understand the statistical behavior of the DBB estimator, we analyze its expectation and variance:

$$\mathbb{E}[\hat{p}_\tau^{\text{dbb}} | p_\tau, \alpha_{\tau-1}, \beta_{\tau-1}] = w \mu_{\tau-1} + (1-w) p_\tau, \quad (14)$$

$$\text{Var}(\hat{p}_\tau^{\text{dbb}} | p_\tau, \alpha_{\tau-1}, \beta_{\tau-1}) = (1-w)^2 \frac{p_\tau(1-p_\tau)}{N}, \quad (15)$$

where $\mu_{\tau-1} = \frac{\alpha_{\tau-1}}{\alpha_{\tau-1} + \beta_{\tau-1}}$ denotes the historical posterior mean and $w = \frac{\lambda(\alpha_{\tau-1} + \beta_{\tau-1})}{\lambda(\alpha_{\tau-1} + \beta_{\tau-1}) + N}$ controls the contribution of past observations.

These show that the DBB estimator introduces bias through shrinkage toward the historical mean, but substantially reduces variance relative to the point estimator (Equation (8)). Importantly, unlike estimated variance based on small-sample rollouts, the posterior variance in DBB cannot collapse to zero, ensuring stable and informative reward signals throughout training. Algorithm 1 summarizes the overall training procedure of the group-based RLVR algorithm with DBB.

3.3. Mean Squared Error of the DBB estimator

To assess the effectiveness of our DBB estimator in predicting the reward distribution, we compute its mean squared error (MSE). To make the estimator's dependence on the underlying reward probabilities $\{p_1, \dots, p_\tau\}$, we re-express its expectation and variance, originally formulated in terms of the posterior parameters $(\alpha_{\tau-1}, \beta_{\tau-1})$.

Based on the update defined in Equation (11), posterior parameters can be expressed as

$$\alpha_\tau = \lambda^\tau \alpha_0 + \sum_{k=1}^{\tau} \lambda^{\tau-k} S_k, \quad \beta_\tau = \lambda^\tau \beta_0 + \sum_{k=1}^{\tau} \lambda^{\tau-k} (N - S_k).$$

For convenience, define the sum of the posterior parameters as H_τ :

$$H_\tau = \alpha_\tau + \beta_\tau = \lambda^\tau (\alpha_0 + \beta_0) + N \sum_{k=1}^{\tau} \lambda^{\tau-k}. \quad (16)$$

Under our formulation (Equation (4)), the number of successful rollouts at epoch k satisfies $S_k \sim \text{Binomial}(N, p_k)$.

Given this, the expectation and variance of the DBB estimator can be written as follows:

$$\mathbb{E}[\hat{p}_\tau^{\text{dbb}} | p_{1:\tau}] = \sum_{k=0}^{\tau} c_k p_k, \quad (17)$$

$$\text{Var}(\hat{p}_\tau^{\text{dbb}} | p_{1:\tau}) = \frac{\sum_{k=1}^{\tau} \lambda^{2(\tau-k)} N p_k (1 - p_k)}{H_\tau^2}, \quad (18)$$

where $p_0 = \frac{\alpha_0}{\alpha_0 + \beta_0}$ is the reward probability of the initial prior, and the weights $c_0 = \frac{\lambda^\tau (\alpha_0 + \beta_0)}{H_\tau}$ and $c_k = \frac{N \lambda^{\tau-k}}{H_\tau}$ are constants. A detailed derivation is provided in Appendix B.

Combining Equations (17) and (18), the MSE of the DBB estimator under non-stationary rewards is

$$\text{MSE}(\hat{p}_\tau^{\text{dbb}} | p_{1:\tau}) = (\mathbb{E}[\hat{p}_\tau^{\text{dbb}} | p_{1:\tau}] - p_\tau)^2 + \text{Var}(\hat{p}_\tau^{\text{dbb}} | p_{1:\tau}). \quad (19)$$

In contrast, the MSE of the point estimator depends only on the current reward distribution

$$\text{MSE}(\hat{p}_\tau^{\text{pt}} | p_\tau) = \frac{p_\tau(1 - p_\tau)}{N}, \quad (20)$$

and is therefore more prone to high variance, especially in low-sample settings.

The bias of the DBB estimator becomes smaller when the underlying reward probabilities evolve gradually, as past values $\{p_k\}_{k < \tau}$ remain close to the current p_τ . In contrast, the variance depends on both the discount factor λ and the number of rollouts N . On the other hand, the point estimator relies solely on p_τ and is therefore more susceptible to high variance, particularly in low-sample settings. Considering these factors, we empirically evaluate the resulting MSE and present the findings in Section 5.2.

4. Experiments

We empirically demonstrate that DBB estimation is effective in the RLVR setting. Section 4.1 describes the models, datasets, baselines, and detailed setup used in our experiments, while Section 4.2 presents the experimental results.

4.1. Experimental Settings

Models & Datasets. We conduct experiments on two model sizes, Qwen3-1.7B-Base and Qwen3-8B-Base (Yang et al., 2025), to investigate the effect of model scales. For both model scales, we use DAPO-Math-17k (Yu et al., 2025a) as the training dataset. As in-distribution evaluation benchmarks for mathematical reasoning, we consider six widely used datasets: MATH500 (Hendrycks et al., 2021), Minerva (Lewkowycz et al., 2022), AIME24/25, AMC24 (Li et al., 2024), and OlympiadBench (Olympiad; He et al., 2024). To further assess out-of-distribution generalization,

we evaluate performance on MMLU-Pro (Wang et al., 2024), GPQA-Diamond (GPQA-D; Rein et al., 2023), and Big-Bench Hard (BBH; Suzgun et al., 2022).

Baselines. The DBB reward estimation aims to improve reward distribution estimation by partially leveraging historical information. To isolate the effect of DBB, we first compare GRPO with the DBB estimator (GRPO-DBB) and naive GRPO, which relies on point estimation. In addition, we include three representative baselines. RePO (Li et al., 2025) is a history-based method that replays full historical trajectories. GRESO (Zheng et al., 2025b) and DAPO (Yu et al., 2025a) are variance-collapse-aware methods: GRESO probabilistically filters prompts likely to lead to variance collapse, while DAPO uses dynamic sampling to discard zero-variance prompts. For all baselines, we follow the best configurations reported in their respective papers. To ensure a fair comparison, we standardize the total number of rollouts used throughout training across all methods.

Training & Evaluation Setups. All training experiments are conducted using the verl¹ framework, and Math-Verify² is employed to extract and verify final answers. Experiments with Qwen3-1.7B-Base and Qwen3-8B-Base are conducted on 4×H200 and 8×H200 GPUs, respectively, except for RePO. Due to the requirement for additional GPU memory, RePO is trained on 8×H200 GPUs for both model scales. The rollout batch size is set to 128, and the gradient update batch size is set to 64. We sample 8 responses per query from the on-policy model and train the model for a total of four epochs. For evaluation, we conduct four independent runs per method with different random seeds, sampling 8 responses per query in each run (i.e., 32 responses per prompt in total). We report Acc@8 as the evaluation metric. Each cell reports the mean and standard deviation across the four runs, and statistical significance is assessed via one-sided paired *t*-tests over per-prompt accuracies. The training setup details for each baseline and the sampling parameters used for validation and evaluation are also provided in Appendix C.

4.2. Main Results

Table 1 reports Acc@8 results across six mathematical reasoning in-distribution benchmarks and three general reasoning out-of-distribution (OOD) benchmarks for five different RLVR algorithms. For GRPO-DBB, the discounting factor λ is set to 0.5 for Qwen3-1.7B-Base and 0.75 for Qwen3-8B-Base; these values are empirically selected based on performance trends under varying λ . Further details are provided in Section 5.2.

¹<https://github.com/verl-project/verl>

²<https://github.com/huggingface/Math-Verify>

Table 1. In-distribution (ID) and out-of-distribution (OOD) evaluation results trained on DAPO-Math-17k. For a fair comparison, all methods are trained with the same total number of rollouts. Evaluation is conducted over four independent runs with Acc@8 per run (i.e., 32 responses per prompt in total); each cell reports mean $\pm$ standard deviation across the four runs. Δ rows report the absolute Acc@8 improvement of GRPO-DBB over each baseline together with a one-sided paired t -test p -value (H_1 : GRPO-DBB $>$ baseline).

Method	In-Distribution							Out-of-Distribution			
	MATH500	Minerva	AIME24	AIME25	AMC24	Olympiad	Avg.	MLLU-Pro	GPQA-D	BBH	Avg.
Qwen3-8B-Base trained with DAPO-Math-17k											
GRPO	87.87 $\pm$ 0.36	39.34 $\pm$ 0.29	29.17 $\pm$ 0.68	25.31 $\pm$ 2.16	57.71 $\pm$ 1.44	54.54 $\pm$ 0.19	48.99 $\pm$ 0.46	50.16 $\pm$ 0.33	45.27 $\pm$ 1.69	59.18 $\pm$ 0.33	51.54 $\pm$ 0.55
RePO	86.22 $\pm$ 0.31	35.02 $\pm$ 0.47	27.29 $\pm$ 0.80	23.44 $\pm$ 1.04	56.25 $\pm$ 0.70	52.70 $\pm$ 0.44	46.82 $\pm$ 0.07	53.29 $\pm$ 0.55	46.34 $\pm$ 1.74	42.44 $\pm$ 0.30	47.35 $\pm$ 0.80
GRESO	87.01 $\pm$ 0.20	33.88 $\pm$ 0.60	27.36 $\pm$ 0.79	22.78 $\pm$ 1.09	57.31 $\pm$ 0.57	52.81 $\pm$ 0.17	46.86 $\pm$ 0.12	62.81 $\pm$ 0.39	48.53 $\pm$ 0.43	65.17 $\pm$ 0.63	58.83 $\pm$ 0.40
DAPO	86.00 $\pm$ 0.07	37.08 $\pm$ 0.30	31.06 $\pm$ 1.19	21.02 $\pm$ 1.14	52.67 $\pm$ 0.92	51.64 $\pm$ 0.14	46.58 $\pm$ 0.16	59.28 $\pm$ 0.67	45.05 $\pm$ 1.79	63.41 $\pm$ 0.29	55.91 $\pm$ 0.46
GRPO-DBB	88.71$\pm$0.18	39.46$\pm$0.51	33.60$\pm$2.05	29.02$\pm$2.53	60.54$\pm$1.00	56.53$\pm$0.25	51.31$\pm$0.53	63.12$\pm$0.38	49.43$\pm$2.11	67.09$\pm$0.13	59.88$\pm$0.66
Δ w.r.t. GRPO	+0.84 (0.0073)	+0.12 (0.2015)	+4.44 (0.0114)	+3.71 (0.0063)	+2.83 (0.0018)	+1.99 (0.0009)	+2.32 (0.0007)	+12.96 (0.0000)	+4.17 (0.0053)	+7.91 (0.0000)	+8.34 (0.0000)
Δ w.r.t. RePO	+2.49 (0.0008)	+4.44 (0.0000)	+6.31 (0.0072)	+5.58 (0.0234)	+4.29 (0.0038)	+3.83 (0.0001)	+4.49 (0.0002)	+9.84 (0.0000)	+3.09 (0.0024)	+24.65 (0.0000)	+12.53 (0.0000)
Δ w.r.t. GRESO	+1.70 (0.0011)	+5.57 (0.0006)	+6.24 (0.0077)	+6.24 (0.0052)	+3.23 (0.0100)	+3.72 (0.0002)	+4.45 (0.0002)	+0.32 (0.0032)	+0.90 (0.1806)	+1.92 (0.0045)	+1.05 (0.0077)
Δ w.r.t. DAPO	+2.71 (0.0000)	+2.37 (0.0011)	+2.54 (0.0220)	+8.00 (0.0010)	+7.87 (0.0012)	+4.89 (0.0001)	+4.73 (0.0001)	+3.84 (0.0013)	+4.39 (0.0019)	+3.68 (0.0001)	+3.97 (0.0001)
Qwen3-1.7B-Base trained with DAPO-Math-17k											
GRPO	67.50 $\pm$ 0.79	25.29 $\pm$ 0.31	9.06 $\pm$ 0.63	4.48 $\pm$ 0.21	23.40 $\pm$ 1.39	29.57 $\pm$ 0.37	26.55 $\pm$ 0.26	33.62 $\pm$ 0.09	21.90 $\pm$ 1.04	12.71 $\pm$ 0.17	22.74 $\pm$ 0.32
RePO	67.27 $\pm$ 0.55	23.78 $\pm$ 0.12	7.60 $\pm$ 1.50	5.94 $\pm$ 0.71	22.57 $\pm$ 1.39	29.00 $\pm$ 0.43	26.03 $\pm$ 0.29	36.58 $\pm$ 0.73	25.25 $\pm$ 0.99	30.37$\pm$0.41	30.73 $\pm$ 0.32
GRESO	67.60 $\pm$ 0.05	23.76 $\pm$ 0.20	8.19 $\pm$ 1.37	6.39 $\pm$ 0.79	23.61 $\pm$ 1.96	30.91 $\pm$ 0.15	26.74 $\pm$ 0.09	35.68 $\pm$ 0.35	27.15 $\pm$ 1.24	26.67 $\pm$ 0.18	29.83 $\pm$ 0.36
DAPO	66.69 $\pm$ 0.13	24.34 $\pm$ 0.23	9.72 $\pm$ 0.86	5.42 $\pm$ 0.90	19.72 $\pm$ 0.45	28.70 $\pm$ 0.06	25.77 $\pm$ 0.33	33.71 $\pm$ 0.69	24.49 $\pm$ 0.86	21.05 $\pm$ 0.12	26.42 $\pm$ 0.42
GRPO-DBB	71.67$\pm$0.22	26.43$\pm$0.67	13.75$\pm$1.13	7.40$\pm$1.04	27.31$\pm$1.90	33.36$\pm$0.50	29.98$\pm$0.28	40.81$\pm$0.11	27.89$\pm$1.46	29.67 $\pm$ 0.89	32.79$\pm$0.79
Δ w.r.t. GRPO	+4.17 (0.0004)	+1.14 (0.0419)	+4.69 (0.0063)	+2.92 (0.0037)	+3.90 (0.0158)	+3.79 (0.0007)	+3.43 (0.0001)	+7.19 (0.0000)	+5.98 (0.0085)	+16.97 (0.0000)	+10.05 (0.0002)
Δ w.r.t. RePO	+4.40 (0.0002)	+2.64 (0.0015)	+6.14 (0.0007)	+1.46 (0.0219)	+4.74 (0.0202)	+4.35 (0.0004)	+3.95 (0.0002)	+4.24 (0.0007)	+2.64 (0.0110)	-0.70 (0.8273)	+2.06 (0.0007)
Δ w.r.t. GRESO	+4.07 (0.0000)	+2.67 (0.0039)	+5.56 (0.0001)	+1.01 (0.0184)	+3.69 (0.0209)	+2.44 (0.0010)	+3.24 (0.0001)	+5.13 (0.0001)	+0.74 (0.0354)	+3.01 (0.0038)	+2.96 (0.0001)
Δ w.r.t. DAPO	+4.98 (0.0000)	+2.08 (0.0031)	+4.03 (0.0119)	+1.98 (0.0387)	+7.58 (0.0018)	+4.66 (0.0001)	+4.22 (0.0002)	+7.11 (0.0002)	+3.39 (0.0119)	+8.62 (0.0001)	+6.38 (0.0004)

For in-distribution evaluation, GRPO-DBB consistently outperforms all four baselines across all benchmarks and model scales. Compared to GRPO, Acc@8 improves by an average of 3.43 points for Qwen3-1.7B-Base and 2.32 points for Qwen3-8B-Base. These improvements are statistically significant in nearly every cell ($p < 0.05$). This indicates that, in the RLVR setting, replacing GRPO’s point estimator with the DBB estimator is more effective for in-distribution downstream tasks. When compared to the other three baselines (RePO, GRESO, and DAPO), GRPO-DBB still achieves average in-distribution Acc@8 gains of 3.24–4.49 (8B) and 2.96–4.22 (1.7B) points. The three baselines exploit historical information in markedly different ways. RePO replays diverse forms of historical information, including tokens, token probabilities, and rewards. GRESO and DAPO instead modify the sampling/filtering strategy. In contrast, GRPO-DBB relies solely on historical reward statistics. Despite this, its strong downstream performance suggests that the DBB estimator utilizes historical information both efficiently and effectively.

For out-of-distribution evaluation, GRPO-DBB consistently outperforms the baselines across nearly all OOD benchmarks and model scales. In particular, it achieves substantial average gains over GRPO of 10.05 and 8.34 points for Qwen3-1.7B-Base and Qwen3-8B-Base, respectively. The only non-significant cell is BBH against RePO on the 1.7B model, where the two methods are statistically indistinguishable. These results demonstrate that the DBB estimation provides a strong advantage in improving generalization, which can be attributed to the Bayesian approach’s ability to account for uncertainty.

Table 2. Effect of the discount factor λ of GRPO-DBB on in-distribution Acc@8 performance. Qwen3-8B-Base achieves the best performance at $\lambda = 0.75$, while Qwen3-1.7B-Base performs best at $\lambda = 0.5$.

Method	MATH500	Minerva	AIME24	AIME25	AMC24	Olympiad	Avg.
Qwen3-8B-Base trained with DAPO 17k							
$\lambda = 1.0$	89.08	37.91	33.33	26.25	59.44	55.19	50.20
$\lambda = 0.75$	88.92	39.48	34.17	30.83	60.00	56.34	51.62
$\lambda = 0.5$	89.35	38.56	33.33	27.08	59.17	56.19	50.61
$\lambda = 0.25$	88.82	39.98	30.83	25.83	60.83	56.34	50.44
GRPO	88.05	39.25	30.00	26.67	56.39	54.82	49.20
Qwen3-1.7B-Base trained with DAPO 17k							
$\lambda = 1.0$	67.30	25.55	9.58	3.33	23.33	28.65	26.29
$\lambda = 0.75$	72.28	25.60	11.25	7.50	26.39	32.68	29.28
$\lambda = 0.5$	71.95	26.31	14.17	7.08	26.94	33.29	29.96
$\lambda = 0.25$	72.22	25.97	14.17	6.25	22.50	32.88	29.00
GRPO	68.37	25.74	8.75	4.58	23.61	29.41	26.74

5. Analysis & Discussion

In this section, we provide an in-depth analysis of the training behavior and estimation properties of GRPO with the DBB reward estimator. We first examine training dynamics to understand how DBB influences optimization stability, exploration behavior, and reward progression over time (Section 5.1). We then conduct ablation studies on the discount factor λ to clarify its role in adapting to different learning regimes (Section 5.2). Next, we empirically analyze the mean squared error (MSE) of DBB relative to point estimation, connecting estimation accuracy to downstream performance (Section 5.3). Finally, we demonstrate that DBB can be readily integrated into alternative advantage formulations, highlighting its generality beyond GRPO (Section 5.4).

5.1. Training Dynamics

Figure 2 illustrates the validation accuracy and training dynamics of GRPO-DBB and GRPO during training. For GRPO-DBB, the discounting factor λ is set to 0.5 for Qwen3-1.7B-Base and 0.75 for Qwen3-8B-Base. For both model scales, GRPO-DBB achieves higher validation accuracy and training rewards than GRPO throughout most of the training process.

Examining the response length over the course of training, we observe that GRPO-DBB consistently produces longer responses than GRPO throughout training. In contrast, the entropy under GRPO grows rapidly, indicating increasingly unstable exploration behavior. Taken together, these observations suggest that GRPO-DBB attains higher rewards by pruning the breadth of response trajectories while allowing exploration to proceed along longer trajectories, corresponding to depth-wise exploration.

Why does DBB shrink entropy and elongate responses?

A decrease in entropy indicates that some token trajectories become less probable and are effectively pruned from the policy’s token tree. This pruning is particularly pronounced for prompts where all sampled rewards within a group are zero. For such prompts, the group-relative advantage of GRPO collapses to zero and yields no training signal. In contrast, DBB still produces a non-zero (negative) advantage that rapidly suppresses incorrect trajectories. We interpret this stronger pruning of zero-variance prompts as a key mechanism underlying the gains of GRPO-DBB. The increase in response length follows naturally. To obtain a reward, the model can explore either (i) breadth-wise by diversifying trajectories or (ii) depth-wise by extending them. Stronger breadth-wise pruning under DBB shifts exploration toward the depth-wise mode, producing significantly longer responses. Importantly, this lower entropy does not compromise sample diversity at evaluation time. As shown in Appendix E, GRPO-DBB matches or exceeds GRPO in Best@8 across both model scales, indicating that meaningful exploratory paths are preserved even as exploration is sharpened.

5.2. Ablation Study on λ

To examine the effect of the discount factor λ in the DBB estimation, we conduct training experiments of GRPO-DBB with $\lambda = 1.0, 0.75, 0.5$, and 0.25 for both model scales.

The hyperparameter λ controls the extent to which the estimation of the Bernoulli parameter p_τ for a given query at epoch τ depends on historical rollout statistics, namely $\alpha_{\tau-1}$ and $\beta_{\tau-1}$. When the learning dynamics are fast and the discrepancy between p_τ and $p_{\tau-1}$ is large, a smaller λ is generally more effective, as it places greater emphasis on recent observations and reduces estimation bias. Conversely,

when the learning dynamics are slower and more stationary, a larger λ tends to yield better performance by incorporating more historical information and thereby reducing estimation variance.

This behavior can be interpreted in conjunction with the training reward dynamics shown in Figure 2. For Qwen3-1.7B-Base, the training reward exhibits sustained changes throughout the training process, indicating continuously evolving estimates of p_τ . In contrast, for Qwen3-8B-Base, the training reward increases rapidly during the very early stage of training (within approximately the first 0.3 epoch) and remains relatively stationary for the remainder of training. As a result, a smaller discounting factor ($\lambda = 0.5$) is more effective for Qwen3-1.7B-Base, while a larger value ($\lambda = 0.75$) better matches the slower and more gradual learning dynamics of Qwen3-8B-Base. More generally, this behavior reflects the bias–variance trade-off of the DBB estimator. As λ increases, variance decreases but bias grows when the reward distribution shifts substantially between epochs. Weaker models or more challenging datasets tend to incur larger such shifts, making a smaller λ preferable. Conversely, once the model fits the training distribution well, the reward distribution becomes nearly stationary and bounded, so a larger λ further reduces MSE. To corroborate this trend, we additionally conduct a λ search on Qwen3-0.6B-Base in Appendix I. There, $\lambda = 0.25$ attains the best performance, consistent with the rule of thumb that *smaller models tend to favor smaller λ* . Based on this observation, we cautiously suggest a starting point for larger models. For 30B+ models trained on datasets of similar difficulty to DAPO-Math-17k, λ close to 1 may be a reasonable starting point, while harder datasets are likely to benefit from a smaller λ .

5.3. Analysis of MSE

To evaluate how accurately the point estimator $\hat{p}_\tau^{\text{pt}}$ and the DBB estimator $\hat{p}_\tau^{\text{dbb}}$ approximate the Bernoulli parameter p_τ during training, we conduct an empirical estimation error analysis. Since obtaining the exact ground-truth value of p_τ would require infinitely many samples, we instead approximate it using the empirical mean of rewards from a finite number of rollouts.

During GRPO training of Qwen3-1.7B-Base, when a randomly selected 10% subset of training prompts $\mathcal{Q}_\tau$ appears in a rollout batch, we additionally sample 128 responses for each such prompt using the policy at that training step. These additional samples are used exclusively for analysis and are not incorporated into training. We then define the empirical mean of the resulting rewards as the reference value $\tilde{p}_\tau(q)$.

$$\tilde{p}_\tau(q) \triangleq \frac{1}{128} \sum_{i=1}^{128} X_{\tau,i}, \quad (21)$$

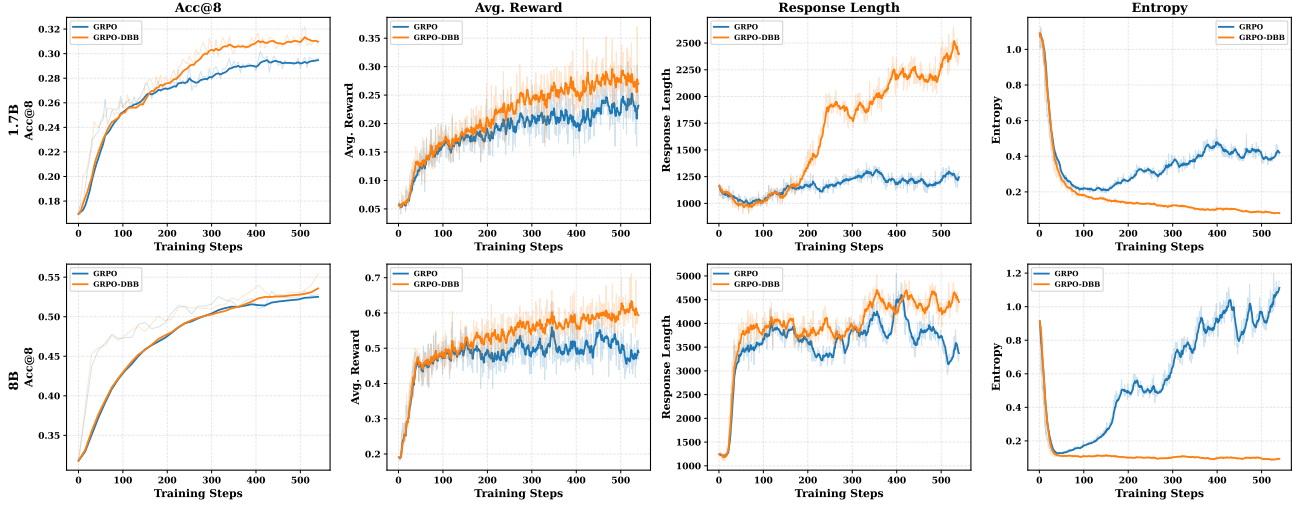


Figure 2. Training dynamics of naive GRPO and GRPO with the DBB estimator (GRPO-DBB) on Qwen3-1.7B-Base (top) and Qwen3-8B-Base (bottom). GRPO-DBB achieves higher validation Acc@8 and training rewards, while maintaining longer responses with controlled entropy compared to GRPO, indicating more stable exploration during training.

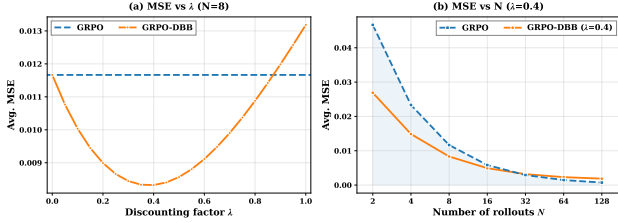


Figure 3. MSE as a function of the discount factor λ and the number of rollouts N . The DBB estimator yields lower MSE than the point estimator across a wide range of λ , and it achieves lower MSE for rollout budgets up to $N = 16$ when $\lambda = 0.4$.

where $o_{\tau,i} \sim \pi_{\theta_{\tau}}(\cdot | q)$ and $X_{\tau,i} = r(o_{\tau,i}, q)$. Since training is conducted for a total of four epochs, this procedure yields a sequence of reference values $\{\hat{p}_{\tau}(q)\}_{\tau=1}^4$ for each query.

Using $\hat{p}_{\tau}(q)$ as a surrogate for the true Bernoulli parameter, we measure the average MSE of the DBB estimator and the point estimator following Equation (19) and (20), respectively.

Effect of λ on MSE Since the value of the DBB estimator depends on the discount factor λ , we empirically analyze how the MSE varies as a function of λ . As shown in Figure 3(a), the MSE attains its minimum at $\lambda = 0.4$, and for most values of λ —except for $\lambda = 0.95$ and $\lambda = 1.0$ —the DBB estimator yields a lower average MSE than the point estimator.

This trend is closely aligned with downstream performance. As reported in Table 2, GRPO-DBB achieves the best performance at $\lambda = 0.5$, while at $\lambda = 1.0$ it performs slightly

Table 3. In-distribution evaluation results on Qwen3-1.7B-Base trained with DAPO-MATH-17k. We apply the proposed discounted Beta-Bernoulli reward distribution estimation to the advantage term of Dr.GRPO and observe consistent performance improvements. Δ denotes the absolute Acc@8 improvement over naive Dr.GRPO.

Method	MATH500	Minerva	AIME24	AIME25	AMC24	Olympiad	Avg.
Qwen3-1.7B-Base trained with DAPO-Math-17k							
Dr.GRPO	69.00	25.14	9.17	7.92	23.89	32.78	27.98
Dr.GRPO-DBB	70.60	26.15	12.08	9.17	28.06	33.59	29.94
Δ w.r.t. Dr.GRPO	+1.60	+1.01	+2.92	+1.25	+4.17	+0.81	+1.96

worse than GRPO with the point estimator. These observations mirror the behavior of the MSE across different values of λ . Taken together, the results indicate that more accurate reward distribution estimation is strongly correlated with improved RLVR performance.

Effect of N on MSE We further examine how the MSE varies as the number of rollouts N increases. Figure 2(b) reports the average MSE as a function of N when $\lambda = 0.4$. The results show that the DBB estimator consistently outperforms the point estimator for N up to 16. This suggests that, in regimes where computational resource constraints necessitate small rollout budgets, the DBB estimator can serve as an effective alternative to point estimation.

5.4. Evaluation on Alternative Advantage Formulations

To examine whether the DBB estimation can be applied to other advantage formulations, we conduct experiments using the advantage term of Dr.GRPO, defined as $A_i = X_i - \mathbb{E}[\hat{p}_{\tau}]$. All experiments are conducted on Qwen3-1.7B-Base with the discounting factor set to $\lambda = 0.5$ for the

Dr.GRPO with the DBB estimation (Dr.GRPO-DBB). As shown in Table 3, Dr.GRPO-DBB consistently improves performance over the naive Dr.GRPO across all in-distribution benchmarks, yielding an average Acc@8 gain of 1.96 points. These results suggest that the DBB estimation is not limited to the advantage term of GRPO, but can be broadly applied to other RLVR algorithms that rely on point estimation, yielding effective and consistent performance gains.

6. Related Work

Reinforcement Learning with Verifiable Rewards. Reinforcement learning with verifiable rewards (RLVR) has emerged as an effective post-training paradigm for improving the reasoning capabilities of large language models by leveraging automatically verifiable reward signals (Lambert et al., 2025; Plaat et al., 2025). In particular, group-based RLVR algorithms, including GRPO and its variants (Shao et al., 2024; Yu et al., 2025a; Liu et al., 2025; Zheng et al., 2025a; Zhao et al., 2025), have demonstrated strong performance on reasoning benchmarks without relying on explicit value models (Yu et al., 2025b).

Variance Collapse in Group-Based RLVR. A well-known limitation of group-based RLVR methods is variance collapse, where the estimated reward variance becomes zero when all sampled responses receive identical rewards, eliminating the training signal and leading to ineffective policy updates (Yu et al., 2025a; Zheng et al., 2025b; Le et al., 2025; Zhang et al., 2025). Prior approaches such as Dynamic sAmpling Policy Optimization (DAPO; Yu et al., 2025a) and GRESO (Zheng et al., 2025b) mitigate this issue by modifying the rollout or sampling strategy. However, these methods either incur substantial additional computational cost or fail to fundamentally eliminate variance collapse due to their probabilistic or heuristic nature.

In contrast, our work addresses variance collapse at the reward estimator level by explicitly modeling uncertainty in the reward distribution. By maintaining a Beta posterior with strictly positive parameters, the DBB process theoretically prevents variance collapse without additional rollouts or changes to the sampling strategy.

Replay- and History-Based RLVR Methods. Replay-based methods aim to improve sample efficiency in RLVR by reusing historical rollouts, as exemplified by RePO (Li et al., 2025) and ExGRPO (Zhan et al., 2025). While potentially effective, these approaches require storing tokens and their probabilities under historical policies and performing additional forward passes, introducing non-trivial memory and computational overhead.

Our approach differs in that it leverages only historical reward statistics rather than full trajectory information, elimi-

nating the need for additional GPU memory or extra forward passes. Empirical results demonstrate that reward signals alone can effectively capture useful historical information for improving RLVR performance.

7. Conclusion

We revisited RLVR from the perspective of reward distribution estimation and identified point estimation under limited rollouts as a key source of sample inefficiency and variance collapse. To this end, we proposed Discounted Beta-Bernoulli (DBB) reward estimation for RLVR, which leverages historical reward information for accurate estimation. Although biased, the DBB estimator achieves substantially lower variance and, under mild non-stationarity of rewards, a lower mean squared error than point estimation; it also theoretically avoids variance collapse and preserves informative training signals without additional rollouts or replay. Experiments across two model scales demonstrate consistent improvements over baselines on all in-distribution and out-of-distribution benchmarks. Our findings highlight reward distribution estimation as a critical yet underexplored component of effective RLVR and suggest that principled estimator design offers a promising direction for improving large-scale reinforcement learning of language models. As future work, we plan to design a dynamic discount factor λ that adapts across different stages of training. We also view DBB as one instance of a broader discounted Bayesian reward-estimation framework. A natural next step is to pair the discounted update with other likelihoods (e.g., a discounted Gaussian) to handle continuous or dense process rewards.

Impact Statements

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Casella, G. and Berger, R. *Statistical inference*. Chapman and Hall/CRC, 2024.
- Gelman, A., Carlin, J. B., Stern, H. S., and Rubin, D. B. *Bayesian data analysis*. Chapman and Hall/CRC, 1995.
- Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- He, C., Luo, R., Bai, Y., Hu, S., Thai, Z. L., Shen, J., Hu, J., Han, X., Huang, Y., Zhang, Y., Liu, J., Qi, L.,

- Liu, Z., and Sun, M. Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems, 2024. URL <https://arxiv.org/abs/2402.14008>.
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the math dataset, 2021. URL <https://arxiv.org/abs/2103.03874>.
- Lambert, N., Morrison, J., Pyatkin, V., Huang, S., Ivison, H., Brahman, F., Miranda, L. J. V., Liu, A., Dziri, N., Lyu, S., Gu, Y., Malik, S., Graf, V., Hwang, J. D., Yang, J., Bras, R. L., Tafford, O., Wilhelm, C., Soldaini, L., Smith, N. A., Wang, Y., Dasigi, P., and Hajishirzi, H. Tulu 3: Pushing frontiers in open language model post-training, 2025. URL <https://arxiv.org/abs/2411.15124>.
- Le, T.-L. V., Jeon, M., Vu, K., Lai, V., and Yang, E. No prompt left behind: Exploiting zero-variance prompts in llm reinforcement learning via entropy-guided advantage shaping, 2025. URL <https://arxiv.org/abs/2509.21880>.
- Lewkowycz, A., Andreassen, A., Dohan, D., Dyer, E., Michalewski, H., Ramasesh, V., Slone, A., Anil, C., Schlag, I., Gutman-Solo, T., Wu, Y., Neyshabur, B., Gur-Ari, G., and Misra, V. Solving quantitative reasoning problems with language models, 2022. URL <https://arxiv.org/abs/2206.14858>.
- Li, J., Beeching, E., Tunstall, L., Lipkin, B., Soletskyi, R., Huang, S., Rasul, K., Yu, L., Jiang, A. Q., Shen, Z., et al. NuminaMath: The largest public dataset in ai4maths with 860k pairs of competition math problems and solutions. *Hugging Face repository*, 13(9):9, 2024.
- Li, S., Zhou, Z., Lam, W., Yang, C., and Lu, C. Repo: Replay-enhanced policy optimization, 2025. URL <https://arxiv.org/abs/2506.09340>.
- Liu, Z., Chen, C., Li, W., Qi, P., Pang, T., Du, C., Lee, W. S., and Lin, M. Understanding rl-zero-like training: A critical perspective, 2025. URL <https://arxiv.org/abs/2503.20783>.
- Plaat, A., Wong, A., Verberne, S., Broekens, J., van Stein, N., and Back, T. Multi-step reasoning with large language models, a survey, 2025. URL <https://arxiv.org/abs/2407.11511>.
- Rein, D., Hou, B. L., Stickland, A. C., Petty, J., Pang, R. Y., Dirani, J., Michael, J., and Bowman, S. R. Gpqa: A graduate-level google-proof q&a benchmark, 2023. URL <https://arxiv.org/abs/2311.12022>.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms, 2017. URL <https://arxiv.org/abs/1707.06347>.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y., Wu, Y., et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Suzgun, M., Scales, N., Schärli, N., Gehrmann, S., Tay, Y., Chung, H. W., Chowdhery, A., Le, Q. V., Chi, E. H., Zhou, D., and Wei, J. Challenging big-bench tasks and whether chain-of-thought can solve them, 2022. URL <https://arxiv.org/abs/2210.09261>.
- Wang, Y., Ma, X., Zhang, G., Ni, Y., Chandra, A., Guo, S., Ren, W., Arulraj, A., He, X., Jiang, Z., Li, T., Ku, M., Wang, K., Zhuang, A., Fan, R., Yue, X., and Chen, W. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark, 2024. URL <https://arxiv.org/abs/2406.01574>.
- Xie, C., Pan, R., Wu, X., Zhang, Y., Fu, J., Gao, T., and Zhou, G. Unlocking exploration in rlvr: Uncertainty-aware advantage shaping for deeper reasoning. *arXiv preprint arXiv:2510.10649*, 2025.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Yu, Q., Zhang, Z., Zhu, R., Yuan, Y., Zuo, X., Yue, Y., Dai, W., Fan, T., Liu, G., Liu, L., Liu, X., Lin, H., Lin, Z., Ma, B., Sheng, G., Tong, Y., Zhang, C., Zhang, M., Zhang, W., Zhu, H., Zhu, J., Chen, J., Chen, J., Wang, C., Yu, H., Song, Y., Wei, X., Zhou, H., Liu, J., Ma, W.-Y., Zhang, Y.-Q., Yan, L., Qiao, M., Wu, Y., and Wang, M. Dapo: An open-source llm reinforcement learning system at scale, 2025a. URL <https://arxiv.org/abs/2503.14476>.
- Yu, T., Ji, B., Wang, S., Yao, S., Wang, Z., Cui, G., Yuan, L., Ding, N., Yao, Y., Liu, Z., Sun, M., and Chua, T.-S. RLpr: Extrapolating rlvr to general domains without verifiers, 2025b. URL <https://arxiv.org/abs/2506.18254>.

Zhan, R., Li, Y., Wang, Z., Qu, X., Liu, D., Shao, J., Wong, D. F., and Cheng, Y. Exgrpo: Learning to reason from experience, 2025. URL <https://arxiv.org/abs/2510.02245>.

Zhang, Y., Yao, W., Yu, C., Liu, Y., Yin, Q., Yin, B., Yun, H., and Li, L. Improving sampling efficiency in rlvr through adaptive rollout and response reuse. *arXiv preprint arXiv:2509.25808*, 2025.

Zhao, Y., Liu, Y., Liu, J., Chen, J., Wu, X., Hao, Y., Lv, T., Huang, S., Cui, L., Ye, Q., Wan, F., and Wei, F. Geometric-mean policy optimization, 2025. URL <https://arxiv.org/abs/2507.20673>.

Zheng, C., Liu, S., Li, M., Chen, X.-H., Yu, B., Gao, C., Dang, K., Liu, Y., Men, R., Yang, A., Zhou, J., and Lin, J. Group sequence policy optimization, 2025a. URL <https://arxiv.org/abs/2507.18071>.

Zheng, H., Zhou, Y., Bartoldson, B. R., Kailkhura, B., Lai, F., Zhao, J., and Chen, B. Act only when it pays: Efficient reinforcement learning for llm reasoning via selective rollouts, 2025b. URL <https://arxiv.org/abs/2506.02177>.

A. Comparison of Statistics Between Point Estimation and Discounted Beta–Bernoulli Estimation

We present Table 4 that summarizes the statistics of point estimation and Discounted Beta–Bernoulli estimation, as discussed in Sections 3.1 and 3.2, in a concise and accessible manner.

Table 4. Comparison between point estimation and DBB estimation for reward distribution modeling. We denote the Bernoulli distribution by $\text{Bern}(\cdot)$.

	Point Estimation	DBB Estimation
$X_{\tau,i}$	$X_{\tau,i} \sim \text{Bern}(p_\tau)$	$X_{\tau,i} \sim \text{Bern}(p_\tau)$
p_τ	fixed but unknown	$p_\tau \sim \text{Beta}(\alpha_\tau, \beta_\tau)$
$\hat{p}_\tau$	$\hat{p}_\tau^{\text{pt}} = \frac{1}{N} \sum_{i=1}^N X_{\tau,i}$	$\hat{p}_\tau^{\text{dbb}} = \frac{\alpha_\tau}{\alpha_\tau + \beta_\tau}$
$\widehat{\text{Var}}(X_\tau)$	$\frac{N}{N-1} \hat{p}_\tau^{\text{pt}} (1 - \hat{p}_\tau^{\text{pt}})$	$\hat{p}_\tau^{\text{dbb}} (1 - \hat{p}_\tau^{\text{dbb}})$
$\mathbb{E}[\hat{p}_\tau]$	p_τ	$w\mu_{\tau-1} + (1-w)p_\tau$
$\text{Bias}(\hat{p}_\tau)$	0	$w(\mu_{\tau-1} - p_\tau)$
$\text{Var}(\hat{p}_\tau)$	$\frac{p_\tau(1-p_\tau)}{N}$	$(1-w)^2 \frac{p_\tau(1-p_\tau)}{N}$

B. Derivation of the Mean, Variance, and MSE of the DBB Estimator

In this section, we provide a detailed derivation of Equations (17) and (18) in the main text. To avoid ambiguity caused by implicit conditioning, we re-express the analysis in terms of the conditional distribution given the sequence of true reward probabilities $\{p_1, p_2, \dots, p_\tau\}$.

B.1. Expansion of Posterior Parameters under Historical Rewards

At each training step k , rollout rewards satisfy

$$S_k \mid p_k \sim \text{Binomial}(N, p_k), \quad X_{k,i} \mid p_k \sim \text{Bernoulli}(p_k), \quad (22)$$

and the DBB updates are given by

$$\alpha_k = \lambda \alpha_{k-1} + S_k, \quad \beta_k = \lambda \beta_{k-1} + (N - S_k). \quad (23)$$

Unrolling the recursion yields

$$\alpha_\tau = \lambda^\tau \alpha_0 + \sum_{k=1}^{\tau} \lambda^{\tau-k} S_k, \quad (24)$$

and the total mass

$$H_\tau \triangleq \alpha_\tau + \beta_\tau = \lambda^\tau (\alpha_0 + \beta_0) + N \sum_{k=1}^{\tau} \lambda^{\tau-k}. \quad (25)$$

Crucially, when N and λ are fixed, H_τ is deterministic and does not depend on the random variables $\{S_k\}_{k=1}^{\tau}$. All stochasticity in the estimator arises from α_τ alone.

B.2. Mean of the DBB Estimator

The DBB estimator is defined as

$$\hat{p}_\tau^{\text{dbb}} = \frac{\alpha_\tau}{H_\tau}. \quad (26)$$

Since H_τ is deterministic,

$$\mathbb{E}[\hat{p}_\tau^{\text{dbb}} | p_{1:\tau}] = \frac{\mathbb{E}[\alpha_\tau | p_{1:\tau}]}{H_\tau}. \quad (27)$$

Using $\mathbb{E}[S_k | p_k] = Np_k$, we obtain

$$\mathbb{E}[\alpha_\tau | p_{1:\tau}] = \lambda^\tau \alpha_0 + N \sum_{k=1}^{\tau} \lambda^{\tau-k} p_k. \quad (28)$$

Therefore,

$$\mathbb{E}[\hat{p}_\tau^{\text{dbb}} | p_{1:\tau}] = \frac{\lambda^\tau \alpha_0 + N \sum_{k=1}^{\tau} \lambda^{\tau-k} p_k}{\lambda^\tau (\alpha_0 + \beta_0) + N \sum_{k=1}^{\tau} \lambda^{\tau-k}} \quad (29)$$

For interpretability, define weights

$$c_0 \triangleq \frac{\lambda^\tau (\alpha_0 + \beta_0)}{H_\tau}, \quad c_k \triangleq \frac{N \lambda^{\tau-k}}{H_\tau}, \quad k = 1, \dots, \tau, \quad p_0 \triangleq \frac{\alpha_0}{\alpha_0 + \beta_0}. \quad (30)$$

Then the estimator mean can be written as

$$\mathbb{E}[\hat{p}_\tau^{\text{dbb}} | p_{1:\tau}] = \sum_{k=0}^{\tau} c_k p_k, \quad \sum_{k=0}^{\tau} c_k = 1, \quad (31)$$

revealing an exponentially weighted average of historical reward probabilities.

B.3. Variance of the DBB Estimator

Again using the determinism of H_τ ,

$$\text{Var}(\hat{p}_\tau^{\text{dbb}} | p_{1:\tau}) = \frac{\text{Var}(\alpha_\tau | p_{1:\tau})}{H_\tau^2}. \quad (32)$$

Since $\{S_k\}$ are independent conditioned on $\{p_k\}$ and $\text{Var}(S_k | p_k) = Np_k(1 - p_k)$,

$$\text{Var}(\alpha_\tau | p_{1:\tau}) = \sum_{k=1}^{\tau} \lambda^{2(\tau-k)} Np_k(1 - p_k). \quad (33)$$

Hence,

$$\text{Var}(\hat{p}_\tau^{\text{dbb}} | p_{1:\tau}) = \frac{\sum_{k=1}^{\tau} \lambda^{2(\tau-k)} Np_k(1 - p_k)}{(\lambda^\tau (\alpha_0 + \beta_0) + N \sum_{k=1}^{\tau} \lambda^{\tau-k})^2} \quad (34)$$

B.4. Bias and Mean Squared Error of the DBB Estimator

By definition,

$$\text{MSE}(\hat{p}_\tau^{\text{dbb}} | p_{1:\tau}) = \text{Bias}^2 + \text{Var}. \quad (35)$$

Using Eq. (31), the conditional bias is

$$\text{Bias}(\hat{p}_\tau^{\text{dbb}} | p_{1:\tau}) = \sum_{k=0}^{\tau-1} c_k (p_k - p_\tau), \quad (36)$$

where the $k = \tau$ term vanishes since $p_\tau - p_\tau = 0$.

Combining Eqs. (34) and (36), we obtain

$$\text{MSE}(\hat{p}_\tau^{\text{dbb}} \mid p_{1:\tau}) = \left(\sum_{k=0}^{\tau-1} c_k(p_k - p_\tau) \right)^2 + \frac{\sum_{k=1}^{\tau} \lambda^{2(\tau-k)} N p_k (1 - p_k)}{H_\tau^2}, \quad (37)$$

where $H_\tau = \lambda^\tau(\alpha_0 + \beta_0) + N \sum_{k=1}^{\tau} \lambda^{\tau-k}$.

C. Implementation Details

Table 5. Common Training hyperparameter settings for all experiments.

Hyperparameter	Qwen3-1.7B-Base	Qwen3-8B-Base
Training Configuration		
Training batch size	128	128
Mini-batch size	64	64
Number of epochs	4	4
Total gradient steps	1080	1080
Samples per prompt	8	8
Max response length	4096	8192
Sampling Configuration		
Training temperature	1.0	1.0
Training top- p	1.0	1.0
Validation temperature	0.6	0.6
Validation top- p	0.95	0.95
Optimization		
Optimizer	AdamW	AdamW
Learning rate	1×10^{-6}	1×10^{-6}
LR warmup steps	0	0
LR scheduler	constant	constant

In this section, we describe the experimental setup details for GRPO-DBB (and more generally any RLVR algorithm that applies the DBB process) and all baseline methods. All training experiments are conducted using the `verl` framework, and `Math-Verify` is employed to extract and normalize final answers from model responses. Training Qwen3-1.7B-Base is performed using 4×H200 GPUs, while Qwen3-8B-Base is trained using 8×H200 GPUs. Due to its substantially higher GPU memory requirements, RePO is trained using 8×H200 GPUs for both model scales.

All training experiments are conducted using the DAPO-Math-17k dataset. For training stability, we filter prompts to a maximum length of 1024 tokens. In addition, to facilitate analysis, we further restrict the dataset so that its size is an integer multiple of the rollout batch size, which is 128. As a result, out of the original 17,391 prompts, 17,280 are used for training.

The hyperparameters shared across all training experiments are summarized in Table 5. Our experimental setup follows configurations that are most commonly adopted in recent RLVR studies. Following DAPO, we remove the KL regularization term during training. Throughout this paper we use PPO clipping in its asymmetric clip-higher form, where the surrogate ratio $w_{i,t}(\theta)$ is clipped to $[1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}]$. For all baselines (GRPO, RePO, GRESO, DAPO, and Dr.GRPO), we use $(\epsilon_{\text{low}}, \epsilon_{\text{high}}) = (0.2, 0.28)$, following DAPO. For GRPO-DBB and Dr.GRPO-DBB, we instead use a wider $(\epsilon_{\text{low}}, \epsilon_{\text{high}}) = (0.98, 0.98)$ to avoid over-clipping the larger advantages produced by the DBB estimator. A detailed analysis showing that this choice does not by itself explain the performance gains is provided in Appendix D.

For GRESO, we follow the original implementation and use the reported best probabilistic-filtering thresholds; for DAPO we use the dynamic-sampling configuration recommended in the original paper, retaining the same total number of rollouts as the other methods. For RePO, which introduces additional hyperparameters and configuration options, we follow the optimal settings recommended in the original paper. Specifically, we set the number of replay samples to 8, use a replay cache size of 16, and adopt the reward-oriented replay strategy.

For both evaluation and validation, we use the sampling parameters specified in Table 5. To reduce training overhead, the validation set is restricted to AIME24/25, AMC24, Minerva, and MATH500. Evaluation on OlympiadBench and

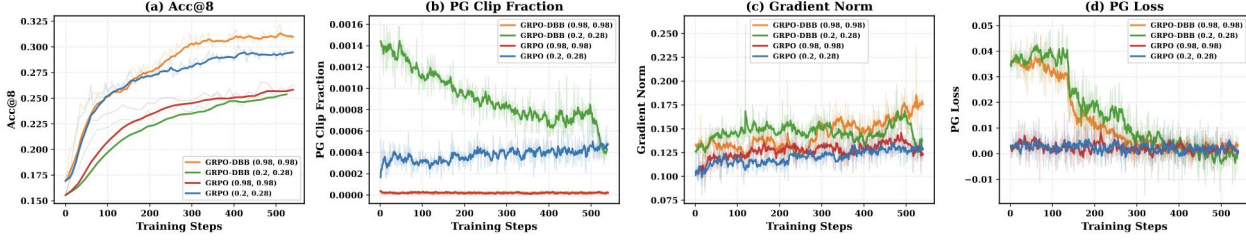


Figure 4. Training dynamics of Qwen3-1.7B-Base with GRPO and GRPO-DBB under clipping ranges (0.2, 0.28) and (0.98, 0.98), respectively. From left to right, the plots show validation Acc@8, policy gradient clip fraction, gradient norm, and policy gradient loss.

Figure 4. Training dynamics of Qwen3-1.7B-Base with GRPO and GRPO-DBB under clipping ranges (0.2, 0.28) and (0.98, 0.98). From left to right: (a) validation Acc@8, (b) policy-gradient clip fraction, (c) gradient norm, and (d) policy-gradient loss. GRPO-DBB suffers excessive clipping under the standard range; once the range is widened, it recovers strong performance. GRPO under the wider range shows no improvement, confirming that the gains of GRPO-DBB are due to the DBB estimator rather than the clip range.

the out-of-distribution benchmarks (MMLU-Pro, GPQA-Diamond, and Big-Bench Hard) is performed only on the best checkpoint selected for each experiment. For statistical significance, we additionally perform four independent evaluation runs per method by varying the evaluation seed; each cell in Table 1 reports the mean and standard deviation across the four runs, and p -values are obtained from one-sided paired t -tests over per-prompt accuracies.

D. Isolating the Effect of the Clipping Range

We further investigate whether the performance gains of GRPO-DBB stem from the DBB estimator itself or from the wider clipping range (0.98, 0.98) used for it.

Why GRPO-DBB requires a wider clip range. The DBB estimator removes GRPO’s zero-sum advantage normalization. As a result, the advantages produced by DBB have a larger dynamic range than those of GRPO, except in the degenerate zero-variance case. Larger advantages scale the same per-token gradients by a larger factor, leading to larger importance ratios. Figure 4(c) and (d) confirm this empirically: GRPO-DBB exhibits larger gradient norms and larger policy-gradient losses than GRPO regardless of the clip range. These effects are therefore *intrinsic to the estimator*, not artifacts of the clip range.

Controls. We train GRPO and GRPO-DBB on Qwen3-1.7B-Base under both $(\epsilon_{\text{low}}, \epsilon_{\text{high}}) = (0.2, 0.28)$ and $(0.98, 0.98)$. Figure 4 compares four diagnostics: (a) validation Acc@8, (b) policy-gradient clip fraction, (c) gradient norm, and (d) policy-gradient loss.

Under the standard (0.2, 0.28) range, GRPO-DBB suffers a 2–3 $\times$ higher clip fraction than GRPO (Figure 4(b)). This suppresses a substantial portion of meaningful training signal and noticeably degrades Acc@8 (Figure 4(a)). Once the clip range is widened to (0.98, 0.98), GRPO-DBB recovers its expected gains and outperforms GRPO. This already shows that the improvements are not merely a consequence of taking larger gradient steps. Critically, GRPO under the same wider (0.98, 0.98) range shows similar or even smaller gradient norms and no improvement in policy gradient loss compared to GRPO under (0.2, 0.28). This rules out the hypothesis that a wider clip range mechanically improves performance. Accordingly, in all main-table experiments we report each method with its own best clipping configuration.

E. Mean@8 and Best@8 Validation Curves

To further characterize the exploration behavior of GRPO-DBB, we plot both Mean@8 (= Acc@8) and Best@8 on the validation set throughout training for both model scales in Figure 5. For Qwen3-1.7B-Base, GRPO-DBB attains higher Best@8 than GRPO for most of training. For Qwen3-8B-Base, GRPO temporarily achieves a higher Best@8 during the middle stage but is eventually overtaken by GRPO-DBB in the final stage. These results indicate that the lower entropy of GRPO-DBB (Figure 2) does not compromise but in fact improves Best@8 in the long run.

To quantify this, we additionally evaluate the best checkpoint of each method on the six in-distribution benchmarks with four independent evaluation seeds (i.e., 32 responses per prompt). GRPO attains Best@8 scores of 61.75 ± 0.92 (8B) and

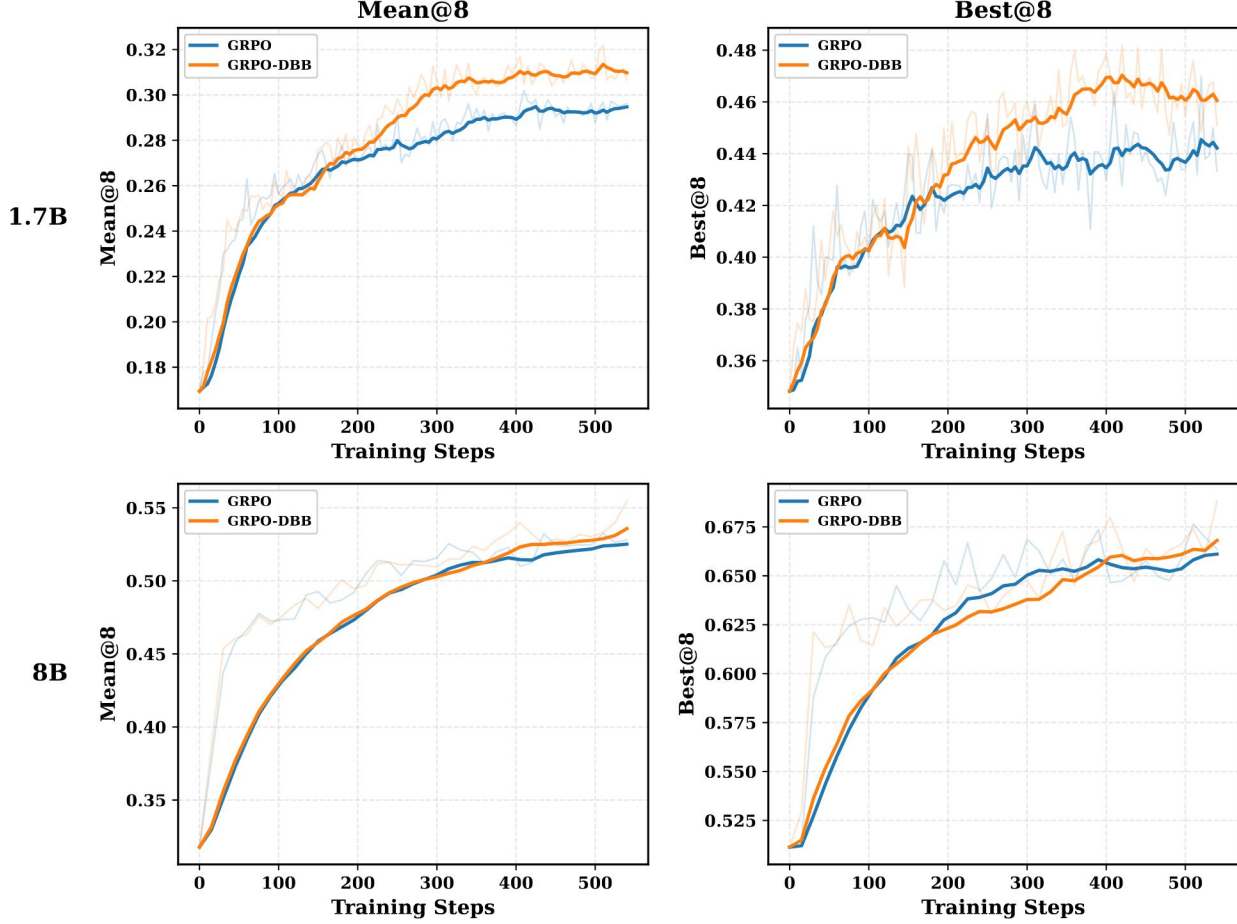


Figure 5. Validation performance of Qwen3-1.7B-Base (top) and Qwen3-8B-Base (bottom) trained with GRPO and GRPO-DBB. The left plots show Mean@8, and the right plots show Best@8 on the validation set.

Figure 5. Validation performance of Qwen3-1.7B-Base (top) and Qwen3-8B-Base (bottom) trained with GRPO and GRPO-DBB. The left column shows Mean@8 (= Acc@8) and the right column shows Best@8 on the validation set. Despite producing lower entropy during training, GRPO-DBB matches or exceeds GRPO in Best@8 across both model scales.

39.79 ± 0.26 (1.7B), while GRPO-DBB attains 63.61 ± 1.16 (8B) and 43.73 ± 0.43 (1.7B). The Best@8 improvements of GRPO-DBB are well outside the standard deviations, so the gains are statistically meaningful. This further indicates that the sharper exploration of GRPO-DBB preserves the diversity of useful response trajectories.

F. Comparison with Per-Prompt EMA and Laplace Smoothing

A natural question is whether simpler shrinkage estimators could achieve similar gains as the DBB estimator. To address this, we additionally derive and evaluate the per-prompt exponential moving average (EMA) and per-prompt Laplace smoothing estimators of the success probability p_τ .

Per-prompt EMA estimator. Let $\hat{p}_\tau^{(\text{EMA})}$ denote the EMA of the per-prompt empirical mean with decay rate λ :

$$\hat{p}_\tau^{(\text{EMA})} = \frac{\sum_{k=1}^{\tau} \lambda^{\tau-k} \hat{p}_k^{\text{pt}}}{\sum_{k=1}^{\tau} \lambda^{\tau-k}}, \quad \hat{p}_k^{\text{pt}} = \frac{S_k}{N}. \quad (38)$$

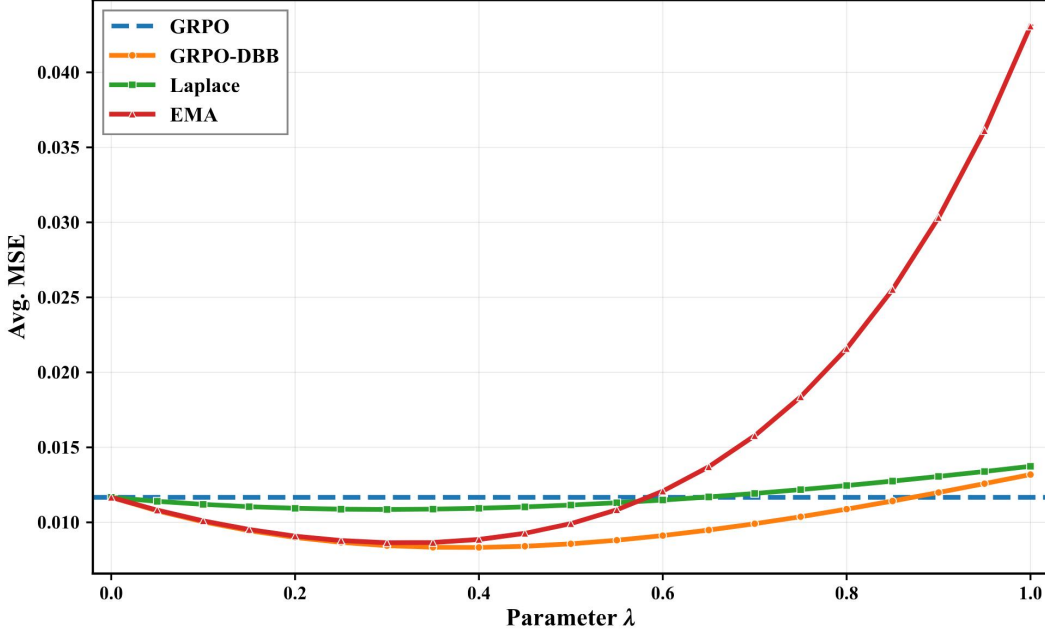


Figure 6. Comparison of MSE as a function of λ for the exponential moving average estimator (EMA), Laplace smoothing estimator (Laplace), point estimator (GRPO), and DBB estimator (GRPO-DBB).

Figure 6. MSE versus λ for per-prompt exponential moving average (EMA), per-prompt Laplace smoothing, the GRPO point estimator (horizontal dashed line), and GRPO-DBB. DBB attains the lowest minimum MSE at $\lambda = 0.4$ and improves on the point estimator over a broader range of λ than EMA or Laplace.

Conditional on the sequence $p_{1:\tau}$ and using $S_k \sim \text{Binomial}(N, p_k)$, its conditional MSE is

$$\text{MSE}(\hat{p}_\tau^{(\text{EMA})} | p_{1:\tau}) = \left(\sum_{k=1}^{\tau} \tilde{c}_k (p_k - p_\tau) \right)^2 + \sum_{k=1}^{\tau} \tilde{c}_k^2 \frac{p_k(1-p_k)}{N}, \quad (39)$$

where $\tilde{c}_k = \lambda^{\tau-k} / \sum_{j=1}^{\tau} \lambda^{\tau-j}$.

Per-prompt Laplace smoothing. With a pseudocount a split symmetrically across the two outcomes, the Laplace-smoothed estimator is

$$\hat{p}_\tau^{(\text{Lap})} = \frac{S_\tau + a}{N + 2a}, \quad (40)$$

which has conditional MSE

$$\text{MSE}(\hat{p}_\tau^{(\text{Lap})} | p_\tau) = \left(\frac{a(1-2p_\tau)}{N+2a} \right)^2 + \frac{N p_\tau(1-p_\tau)}{(N+2a)^2}. \quad (41)$$

We parameterize the pseudocount as $a = \lambda \cdot a_{\max}$ to keep a comparable λ sweep range against EMA and DBB.

Empirical comparison. Following the protocol of Section 5.2 (Qwen3-1.7B-Base, 128-rollout reference), we sweep λ for each estimator. As shown in Figure 6, GRPO-DBB achieves the lowest MSE of 0.0083 at $\lambda = 0.4$, outperforming all other methods. EMA attains its best MSE of 0.0086 at $\lambda = 0.3$ (+2.67% over DBB) and degrades rapidly as λ grows, indicating high sensitivity to λ . Laplace smoothing attains its best MSE of 0.0108 at $\lambda = 0.3$ (+21.71% over DBB) and also outperforms point estimation only within a narrow λ range. Overall, DBB is the most favorable option among these light-weight shrinkage estimators. It attains the lowest minimum MSE and maintains a broader λ range over which it improves on point estimation.

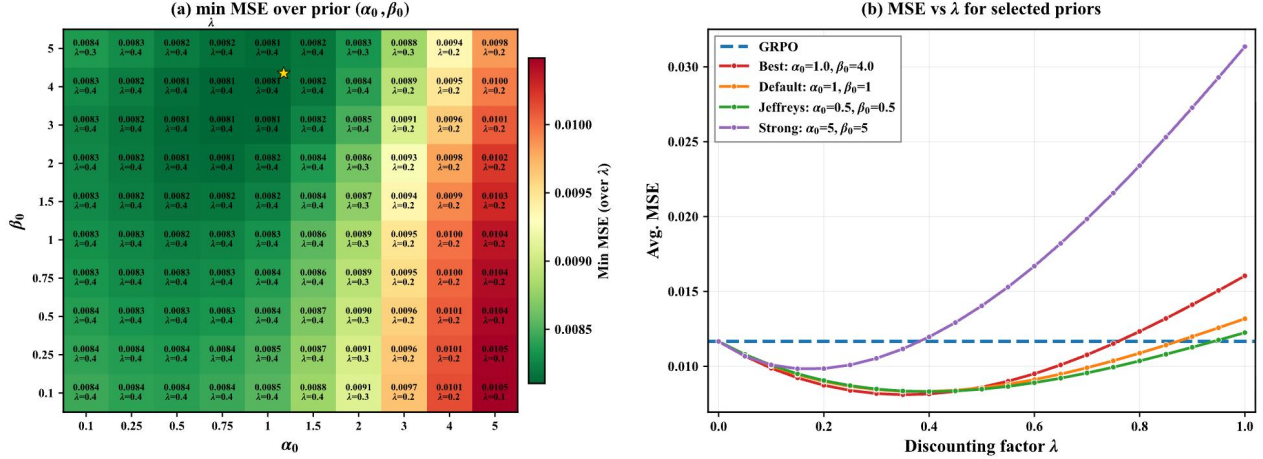


Figure 7. (a) shows a heatmap of $\min_{\lambda}$ MSE over (α_0, β_0) , and (b) presents the MSE as a function of λ for selected (α_0, β_0) pairs.

Figure 7. (a) Heatmap of $\min_{\lambda}$ MSE of the DBB estimator over (α_0, β_0) . (b) MSE versus λ for selected priors. The default symmetric prior $(\alpha_0, \beta_0) = (1, 1)$ is competitive with the best prior and robust over a wide range of λ .

G. Sensitivity to the Prior Hyperparameters (α_0, β_0)

We additionally measure the MSE of the DBB estimator with respect to the prior hyperparameters (α_0, β_0) and the discount factor λ under the same setup as Section 5.2.

Figure 7(a) shows a heatmap of $\min_{\lambda}$ MSE of the DBB estimator over (α_0, β_0) . For most reasonable prior choices, the achievable MSE remains close to the optimum. Performance degrades only when $\alpha_0 + \beta_0$ is large, where a strong prior diminishes the influence of the N on-policy rewards. In that regime, the estimator becomes more sensitive to λ , which can amplify bias and inflate MSE (Figure 7(b)).

In practice we initialize the Beta posterior with the symmetric uninformative prior $(\alpha_0, \beta_0) = (1, 1)$. This choice is both reasonable and robust. In the absence of prior knowledge about p_{τ} , symmetric initialization avoids injecting bias toward either outcome. At the same time, it keeps the prior mass small relative to typical batch sizes. As Figure 7(b) shows, $(1, 1)$ achieves a relatively low MSE across a wide range of λ and a competitive minimum. Overall, MSE is sensitive to (α_0, β_0) and to the corresponding λ , but this sensitivity is not a practical concern as long as pathological prior choices are avoided.

H. Hierarchical / Empirical-Bayes Prior

We also consider an empirical Bayes (EB) variant of DBB as a representative way to introduce a hierarchical / shared prior across prompts. The shared Beta hyperparameters $(\alpha_{\text{EB}}(s), \beta_{\text{EB}}(s))$ are estimated online via method-of-moments from the population of per-prompt posteriors at step s . For a given prompt with N on-policy rewards summing to S_{τ} , the EB estimator is

$$\hat{p}_{\tau}^{(\text{EB})} = \frac{S_{\tau} + \hat{\alpha}_{\text{EB}}(s)}{N + \hat{\alpha}_{\text{EB}}(s) + \hat{\beta}_{\text{EB}}(s)}. \quad (42)$$

This estimator does not require a manually tuned discount factor: the shared prior already provides cross-prompt shrinkage.

Following the same protocol as Section 5.2, Figure 8 compares the MSE of the EB estimator, the point estimator (GRPO), and the DBB estimator over λ . EB sits between point estimation and the optimum of DBB. This indicates a trade-off: EB avoids manual λ tuning at the cost of a higher minimum MSE, while DBB attains a strictly lower MSE when λ is chosen reasonably. Combining a hierarchical prior with the discounted update of DBB is a natural direction for future work.

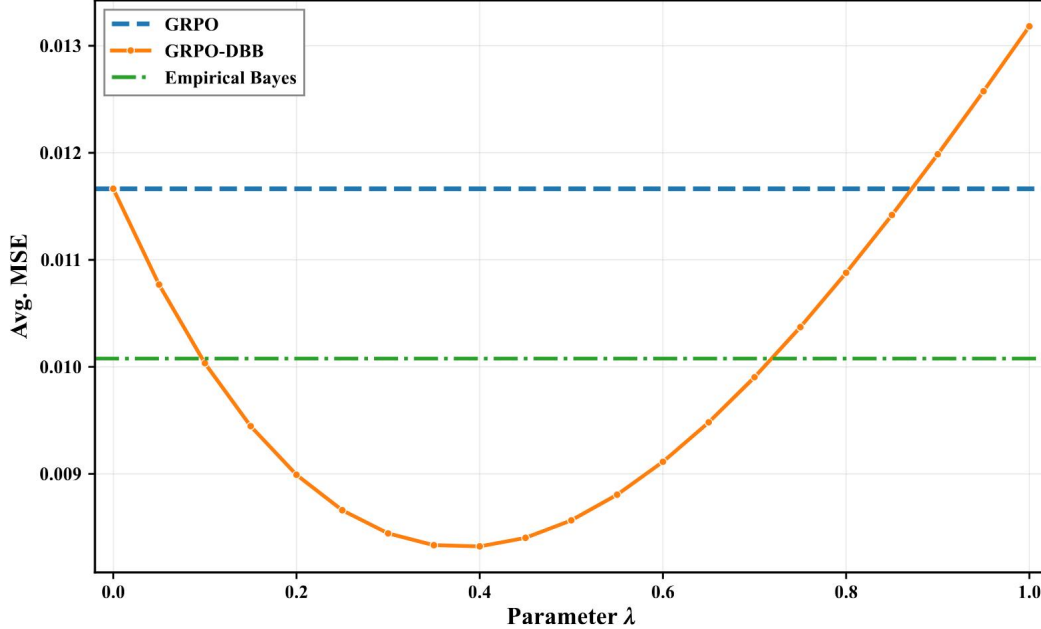


Figure 8. Comparison of MSE as a function of λ for the Empirical Bayes estimator (Empirical Bayes) obtained via online method-of-moments, the point estimator (GRPO), and the DBB estimator (GRPO-DBB).

Figure 8. MSE versus λ for the empirical Bayes (EB) estimator (online method-of-moments), the GRPO point estimator (dashed), and GRPO-DBB. EB improves over the point estimator without per-prompt tuning, but DBB attains a lower minimum MSE.

I. Discount-Factor Search on Qwen3-0.6B-Base

To further investigate how the optimal discount factor λ depends on model scale, we conduct an additional λ search on a smaller backbone, Qwen3-0.6B-Base, trained with DAPO-Math-17k. Table 6 reports Acc@8 on the six in-distribution benchmarks for $\lambda \in \{0.25, 0.5, 0.75, 1.0\}$.

The optimal λ for the 0.6B model is 0.25, smaller than the optimal values $\lambda^* = 0.5$ for 1.7B and $\lambda^* = 0.75$ for 8B. Together with our analysis in Section 5.2, this supports a simple rule of thumb: *smaller (or weaker) backbones tend to favor smaller λ* . The reason is that their reward distribution shifts more rapidly during training, so the bias of larger λ outweighs the variance reduction it provides. We extrapolate this rule cautiously to larger models. For 30B+ models trained on datasets of comparable difficulty to DAPO-Math-17k, λ close to 1 is a reasonable starting point. Harder datasets (where shifts are larger) are likely to benefit from a smaller λ .

Table 6. Effect of the discount factor λ of GRPO-DBB on in-distribution Acc@8 performance for Qwen3-0.6B-Base trained on DAPO-Math-17k. The best average performance is attained at $\lambda = 0.25$.

Method	MATH500	MinervaMath	AIME24	AIME25	AMC24	Avg.
Qwen3-0.6B-Base trained with DAPO-Math-17k						
$\lambda = 1.0$	47.12	14.25	2.92	0.00	10.83	15.02
$\lambda = 0.75$	48.05	14.52	2.08	0.42	10.28	15.07
$\lambda = 0.5$	47.95	14.11	2.08	1.25	11.94	15.47
$\lambda = 0.25$	49.98	14.15	3.33	1.67	8.61	15.55